DS – HOMEWORK 3

1. Store n elements using dynamic memory allocation and find the largest number

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int n, i, largest;
   int *arr;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   arr = (int *)malloc(n * sizeof(int));

   if (arr == NULL)
   {
      printf("Memory allocation failed.");
      return 1;
   }

   printf("Enter %d elements:\n", n);

   for (i = 0; i < n; i++)
      scanf("%d", &arr[i]);
```

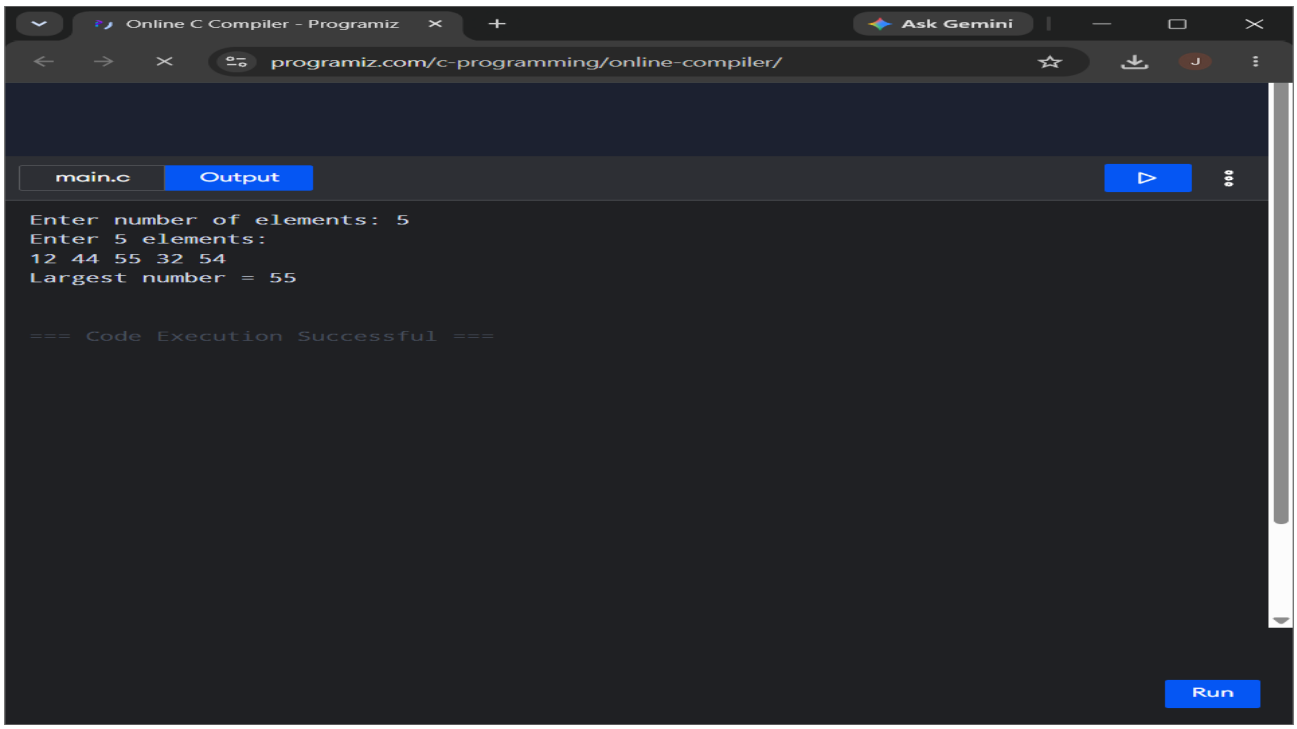
```
    largest = arr[0];

    for (i = 1; i < n; i++)
    {
        if (arr[i] > largest)
            largest = arr[i];
    }

    printf("Largest number = %d\n", largest);

    free(arr);

    return 0;
}
```



2. Print an array in reverse order using pointers

```c
#include <stdio.h>

int main()
{
   int arr[100], n, i;
   int *ptr;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   printf("Enter array elements:\n");

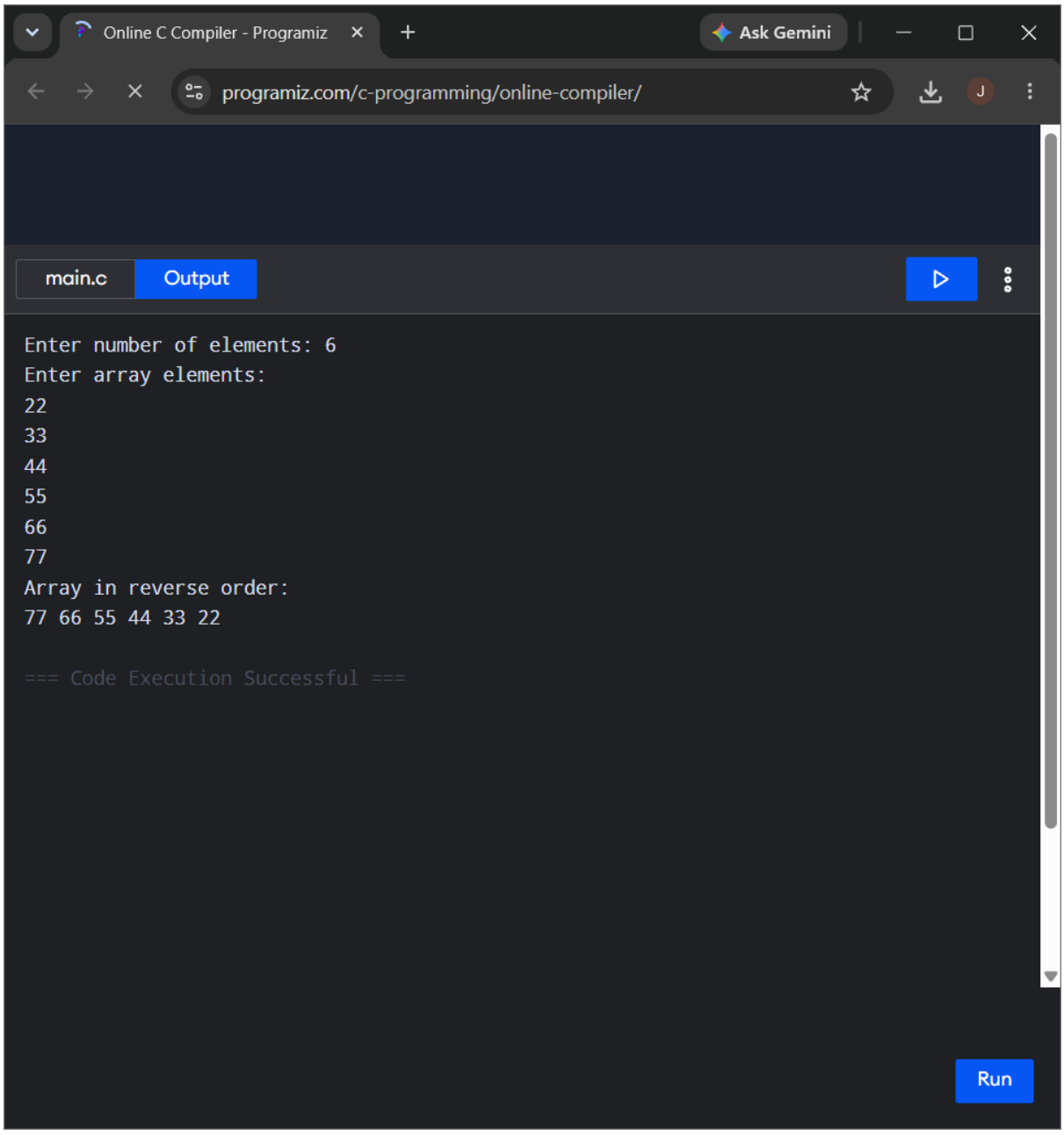
   for (i = 0; i < n; i++)
      scanf("%d", &arr[i]);

   ptr = &arr[n - 1];

   printf("Array in reverse order:\n");

   for (i = n - 1; i >= 0; i--)
   {
      printf("%d ", *ptr);
      ptr--;
   }

   return 0;
}
```



3. Store student information using structure and sort by name

#include <stdio.h>

#include <string.h>

struct Student

```
{
    int roll;
    char name[50];
    float marks;
};

int main()
{
    struct Student s[100], temp;
    int n, i, j;

    printf("Enter number of students: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("\nEnter details of student %d:\n", i + 1);

        printf("Roll number: ");
        scanf("%d", &s[i].roll);

        printf("Name: ");
        scanf(" %[^\n]", s[i].name);

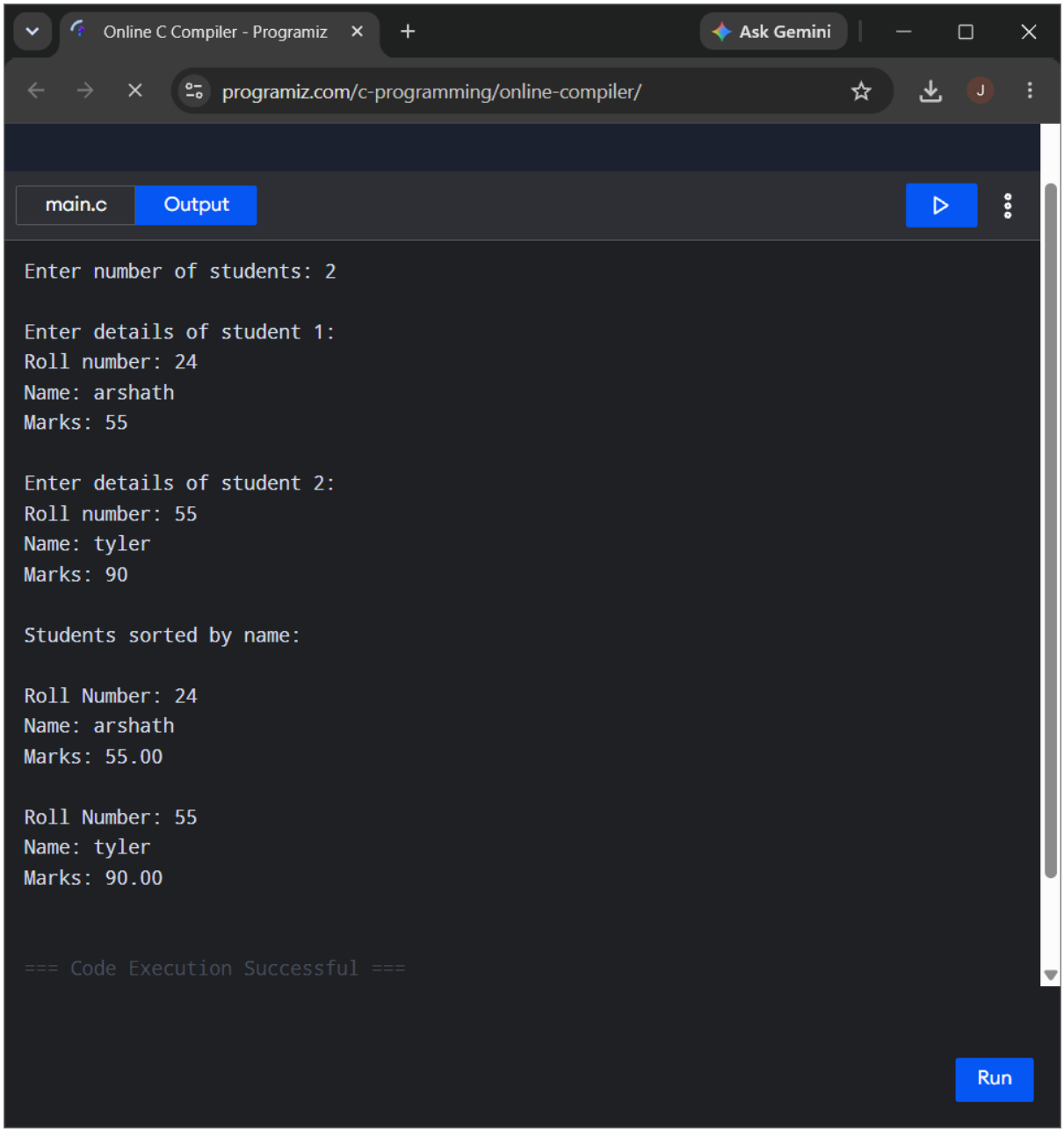
        printf("Marks: ");
        scanf("%f", &s[i].marks);
    }
```

```
    /* Sorting students by name */
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (strcmp(s[i].name, s[j].name) > 0)
            {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    printf("\nStudents sorted by name:\n");

    for (i = 0; i < n; i++)
    {
        printf("\nRoll Number: %d", s[i].roll);
        printf("\nName: %s", s[i].name);
        printf("\nMarks: %.2f\n", s[i].marks);
    }

    return 0;
}
```



4. Add n distances using structures

```
#include <stdio.h>


struct Distance
{
   int feet;
   int inches;
```

```
};

int main()
{
   struct Distance d[100], total = {0, 0};
   int n, i;

   printf("Enter number of distances: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++)
   {
      printf("\nEnter distance %d:\n", i + 1);

      printf("Feet: ");
      scanf("%d", &d[i].feet);

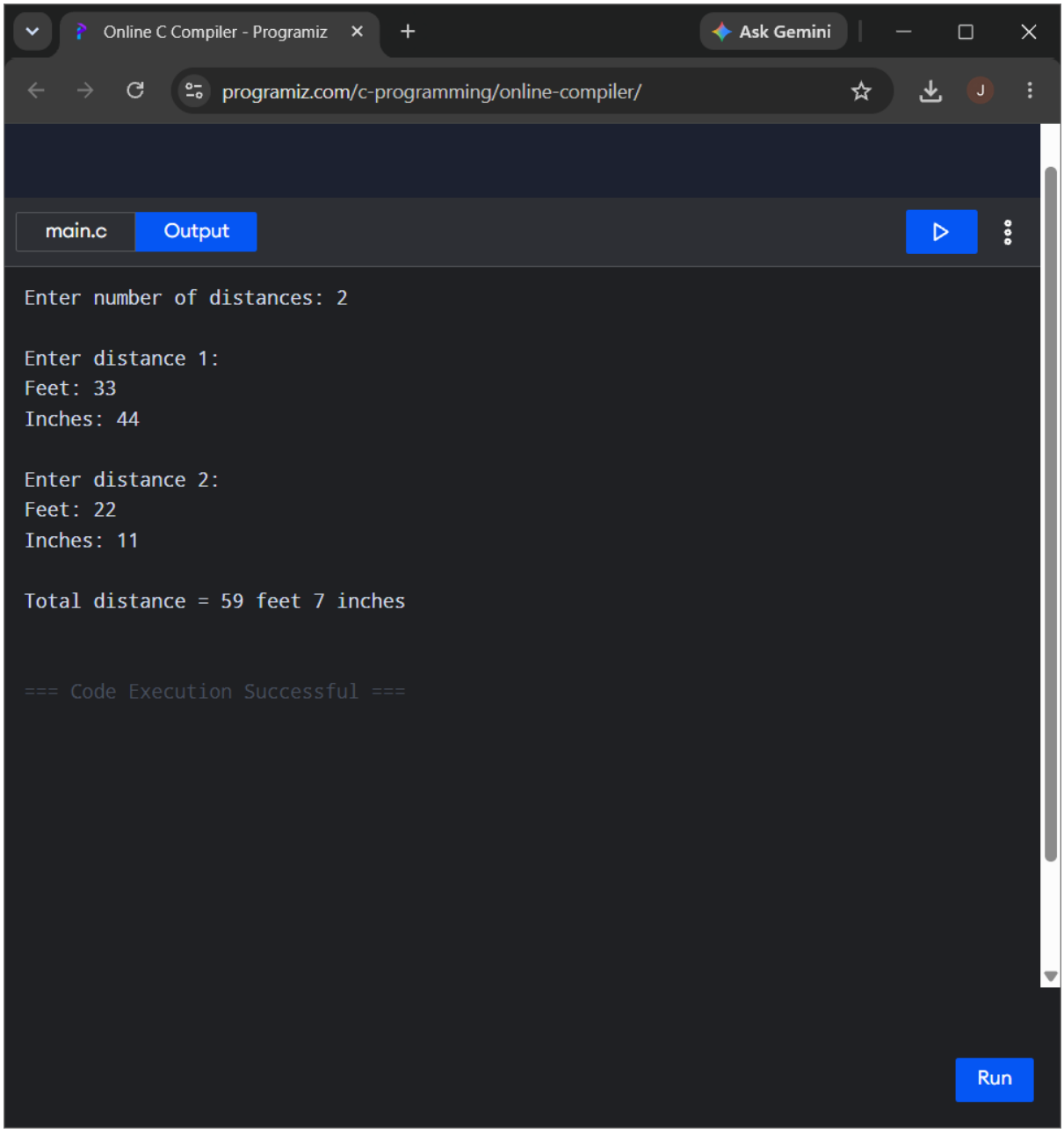
      printf("Inches: ");
      scanf("%d", &d[i].inches);

      total.feet += d[i].feet;
      total.inches += d[i].inches;
   }

   /* Convert inches into feet */
   total.feet += total.inches / 12;
   total.inches = total.inches % 12;
```

```
    printf("\nTotal distance = %d feet %d inches\n",
           total.feet, total.inches);

    return 0;
}
```



5. Add two complex numbers by passing structures to a function

```
#include <stdio.h>
```

```
struct Complex
{
   float real;
   float imag;
};

struct Complex add(struct Complex c1, struct Complex c2)
{
   struct Complex result;

   result.real = c1.real + c2.real;
   result.imag = c1.imag + c2.imag;

   return result;
}

int main()
{
   struct Complex c1, c2, sum;

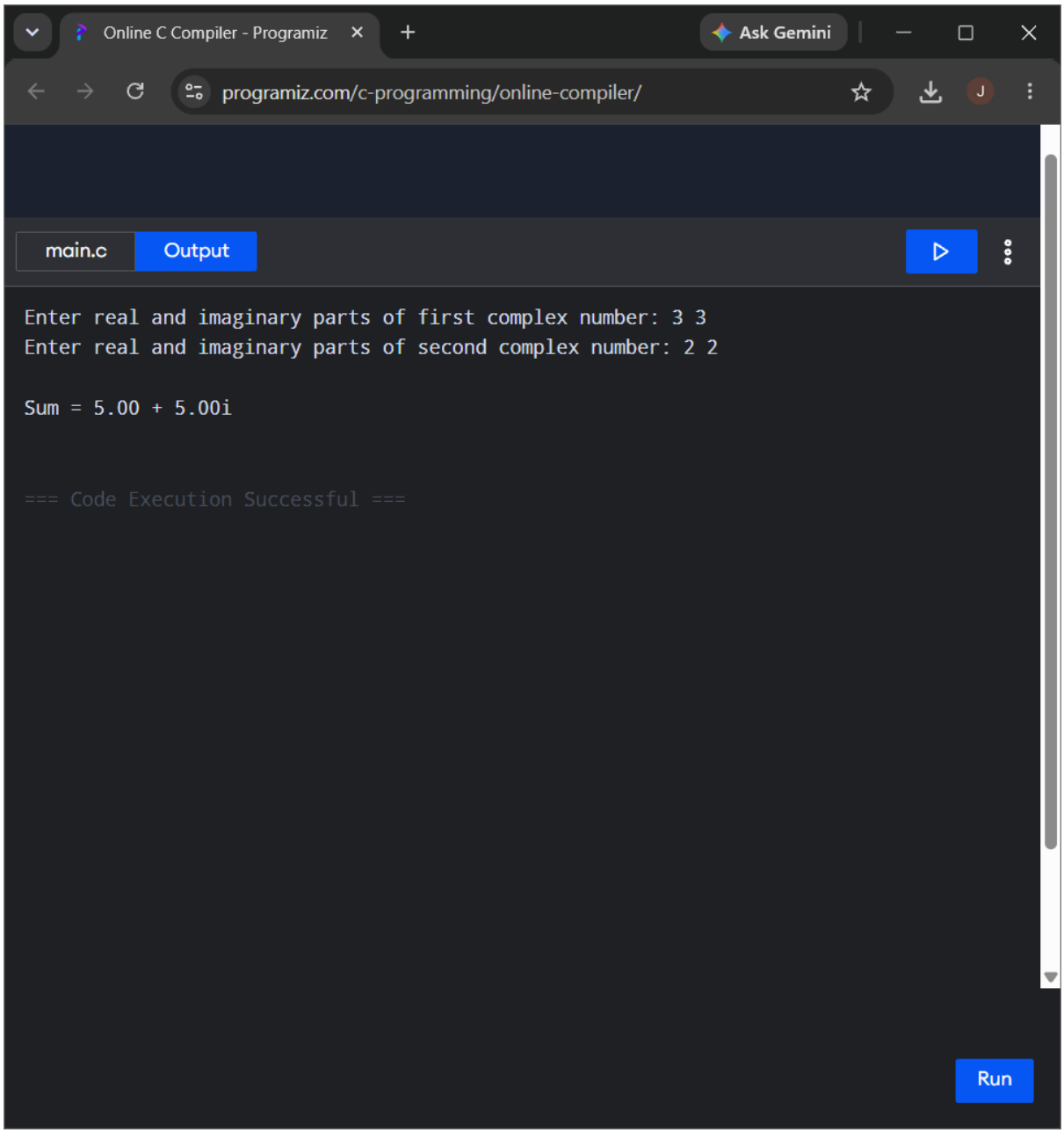
   printf("Enter real and imaginary parts of first complex number: ");
   scanf("%f %f", &c1.real, &c1.imag);

   printf("Enter real and imaginary parts of second complex number: ");
   scanf("%f %f", &c2.real, &c2.imag);

   sum = add(c1, c2);
```

```
    printf("\nSum = %.2f + %.2fi\n", sum.real, sum.imag);

    return 0;
}
```



6. Library management system using structures

```
#include <stdio.h>
#include <string.h>

struct Book
{
   int id;
   char title[50];
   char author[50];
   int available;
};

int main()
{
   struct Book books[100];
   int n, i, choice, searchId;

   printf("Enter number of books: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++)
   {
      printf("\nEnter details of book %d:\n", i + 1);

      printf("Book ID: ");
      scanf("%d", &books[i].id);

      printf("Title: ");
      scanf(" %[^\n]", books[i].title);
```

```
    printf("Author: ");
    scanf(" %[^\n]", books[i].author);

    books[i].available = 1;
}

do
{
    printf("\n--- Library Menu ---\n");
    printf("1. Display books\n");
    printf("2. Search book\n");
    printf("3. Issue book\n");
    printf("4. Return book\n");
    printf("5. Exit\n");

    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 1)
    {
        printf("\nBook Details:\n");

        for (i = 0; i < n; i++)
        {
            printf("\nID: %d", books[i].id);
            printf("\nTitle: %s", books[i].title);
            printf("\nAuthor: %s", books[i].author);
```

```
        printf("\nStatus: %s\n",
            books[i].available ? "Available" : "Issued");
    }
}
else if (choice == 2)
{
    printf("Enter Book ID to search: ");
    scanf("%d", &searchId);

    for (i = 0; i < n; i++)
    {
        if (books[i].id == searchId)
        {
            printf("\nBook found!");
            printf("\nTitle: %s", books[i].title);
            printf("\nAuthor: %s\n", books[i].author);
            break;
        }
    }

    if (i == n)
        printf("Book not found.\n");
}
else if (choice == 3)
{
    printf("Enter Book ID to issue: ");
    scanf("%d", &searchId);
```

```
    for (i = 0; i < n; i++)
    {
        if (books[i].id == searchId)
        {
            if (books[i].available)
            {
                books[i].available = 0;
                printf("Book issued successfully.\n");
            }
            else
                printf("Book is already issued.\n");

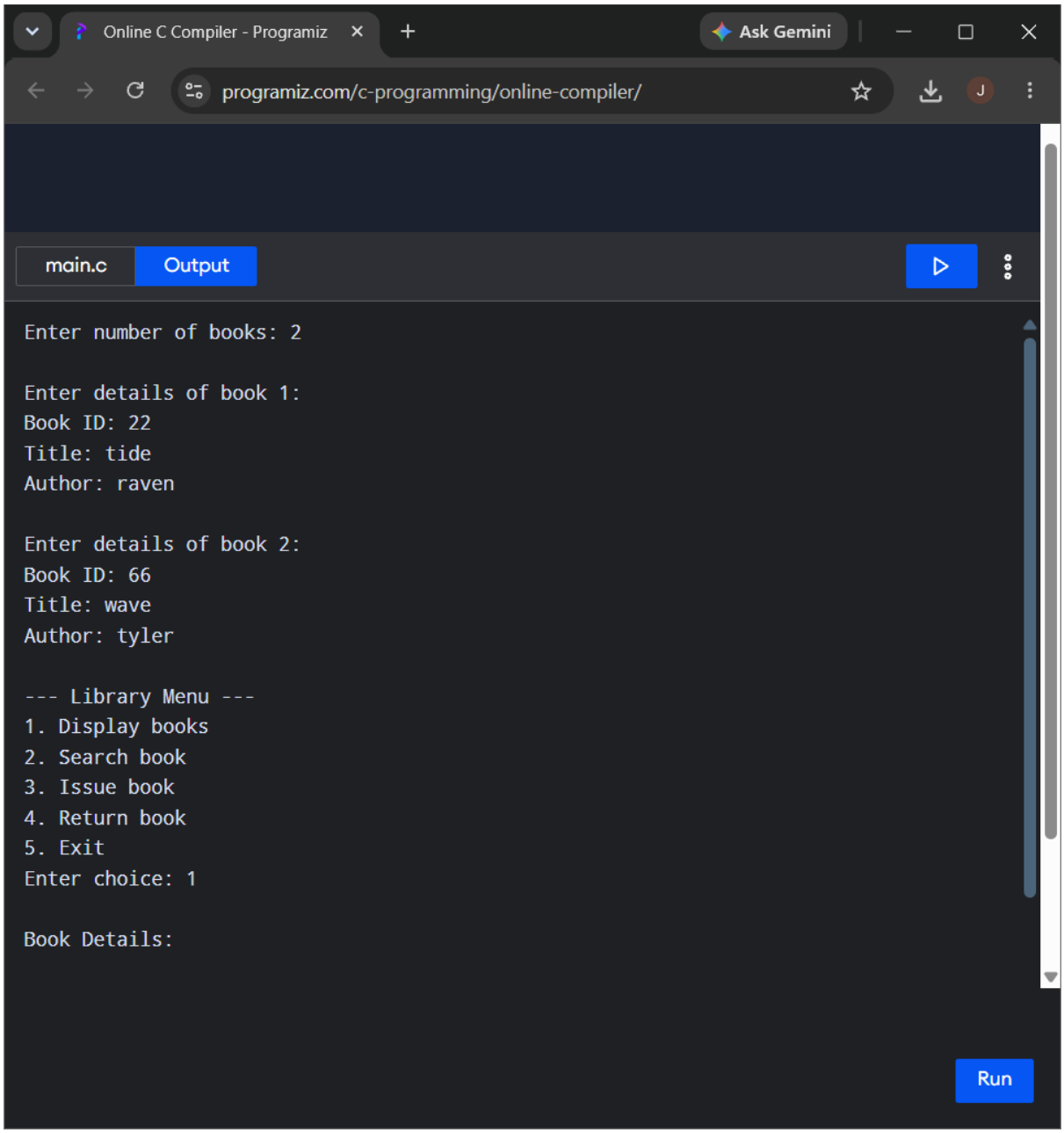
            break;
        }
    }
}
else if (choice == 4)
{
    printf("Enter Book ID to return: ");
    scanf("%d", &searchId);

    for (i = 0; i < n; i++)
    {
        if (books[i].id == searchId)
        {
            books[i].available = 1;
            printf("Book returned successfully.\n");
            break;
```

```
            }
        }
    }

    } while (choice != 5);

    return 0;
}
```



7. Decimal to binary, octal and hexadecimal conversion

```
#include <stdio.h>

void binary(int n)
{
   int a[32], i = 0;

   if (n == 0)
   {
      printf("0");
      return;
   }

   while (n > 0)
   {
      a[i] = n % 2;
      n = n / 2;
      i++;
   }

   while (i > 0)
   {
      printf("%d", a[--i]);
   }
}

int main()
```

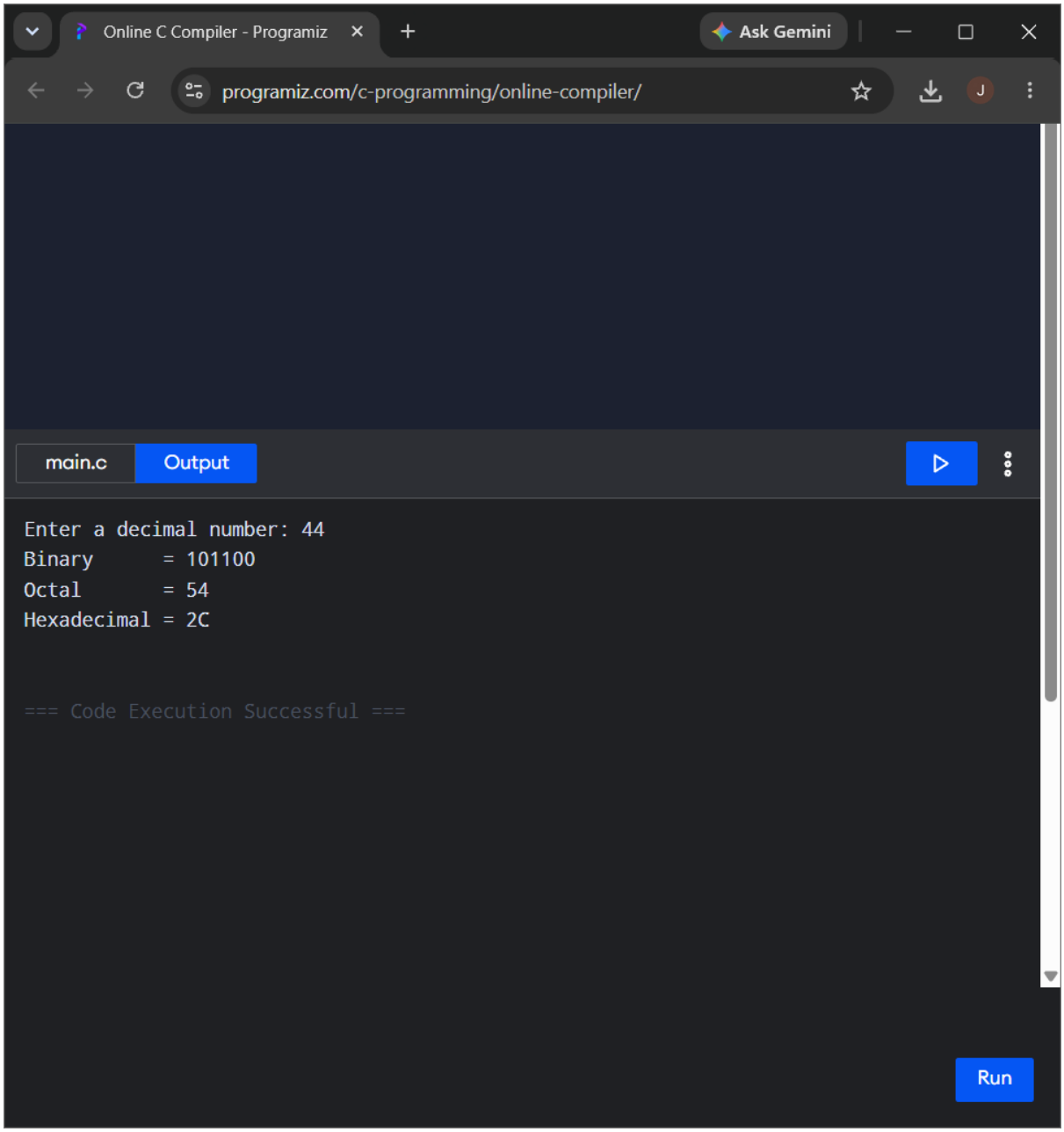
```
{
   int n;

   printf("Enter a decimal number: ");
   scanf("%d", &n);

   printf("Binary      = ");
   binary(n);

   printf("\nOctal       = %o", n);
   printf("\nHexadecimal = %X\n", n);

   return 0;
}
```



8. Calculate difference between two time periods

```
#include <stdio.h>

struct Time
{
   int hours;
   int minutes;
```

```
    int seconds;
};

int main()
{
    struct Time t1, t2, diff;
    int sec1, sec2, difference;

    printf("Enter first time (hours minutes seconds): ");
    scanf("%d %d %d", &t1.hours, &t1.minutes, &t1.seconds);

    printf("Enter second time (hours minutes seconds): ");
    scanf("%d %d %d", &t2.hours, &t2.minutes, &t2.seconds);

    sec1 = t1.hours * 3600 + t1.minutes * 60 + t1.seconds;
    sec2 = t2.hours * 3600 + t2.minutes * 60 + t2.seconds;

    difference = sec1 - sec2;

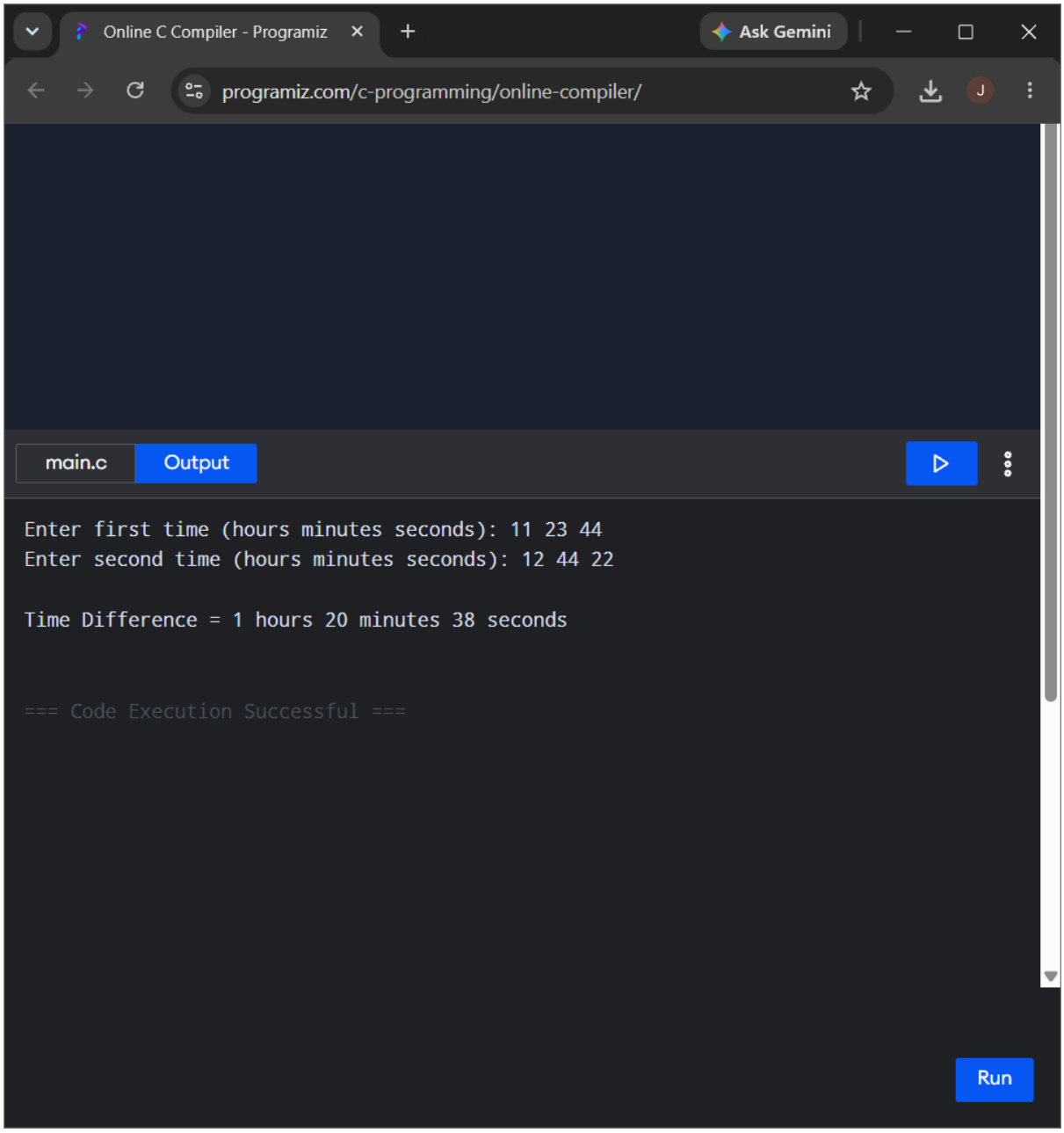
    if (difference < 0)
        difference = -difference;

    diff.hours = difference / 3600;
    difference = difference % 3600;

    diff.minutes = difference / 60;
    diff.seconds = difference % 60;
```

```
    printf("\nTime Difference = %d hours %d minutes %d seconds\n",
        diff.hours, diff.minutes, diff.seconds);

    return 0;
}
```



9. Employee management system using structures

```
#include <stdio.h>
```

```
struct Employee
{
   int id;
   char name[50];
   float salary;
};

int main()
{
   struct Employee emp[100];
   int n, i;

   printf("Enter number of employees: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++)
   {
      printf("\nEnter details of employee %d:\n", i + 1);

      printf("Employee ID: ");
      scanf("%d", &emp[i].id);

      printf("Name: ");
      scanf(" %[^\n]", emp[i].name);

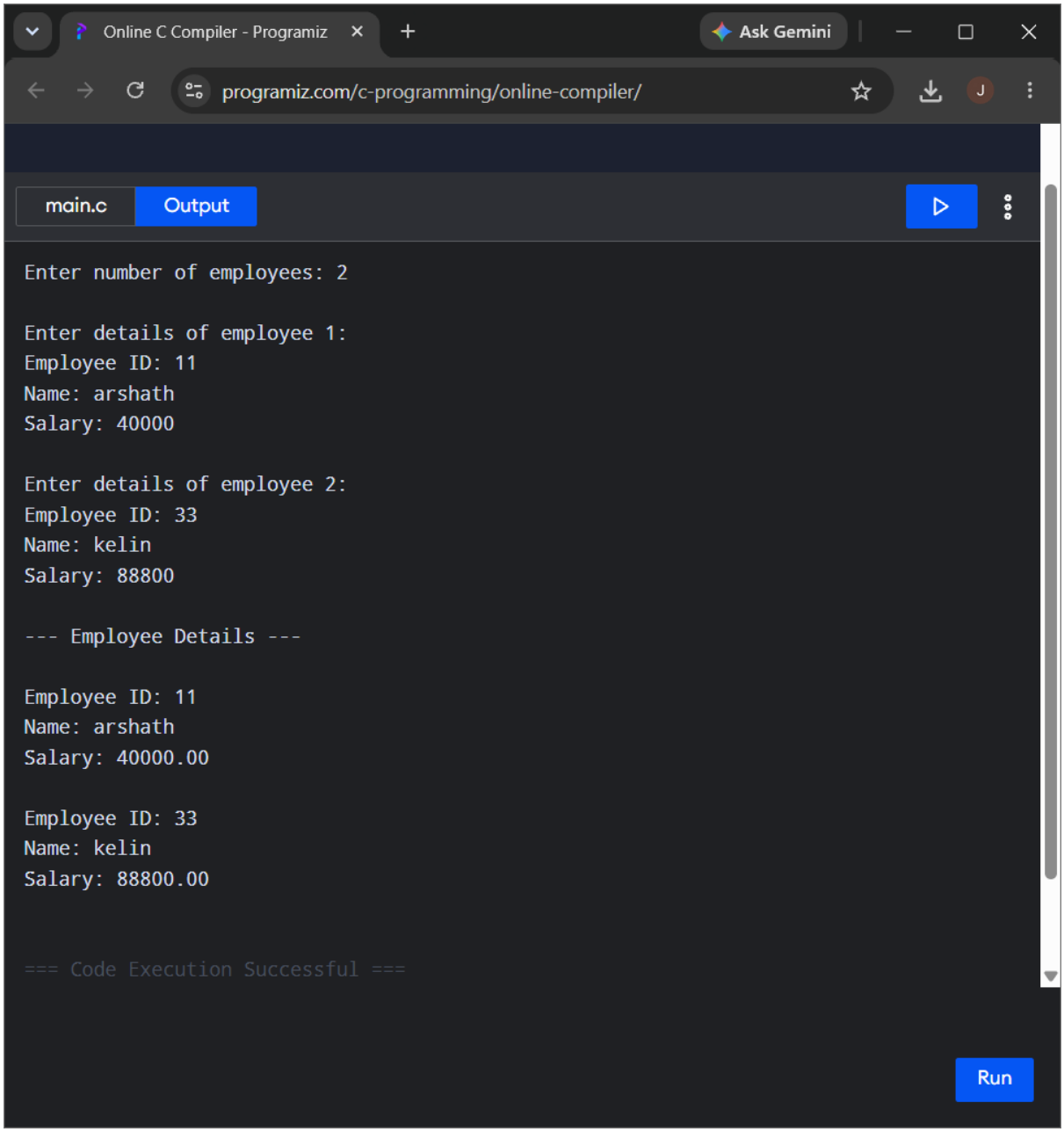
      printf("Salary: ");
      scanf("%f", &emp[i].salary);
```

```
    }

    printf("\n--- Employee Details ---\n");

    for (i = 0; i < n; i++)
    {
        printf("\nEmployee ID: %d", emp[i].id);
        printf("\nName: %s", emp[i].name);
        printf("\nSalary: %.2f\n", emp[i].salary);
    }

    return 0;
}
```



10. Calculate area and perimeter of rectangle and triangle using function pointers

```
#include <stdio.h>

#include <math.h>


/* Rectangle functions */

float rectangleArea(float l, float b)

{
```

```
    return l * b;
}

float rectanglePerimeter(float l, float b)
{
    return 2 * (l + b);
}

/* Triangle functions */
float triangleArea(float a, float b, float c)
{
    float s = (a + b + c) / 2;
    return sqrt(s * (s - a) * (s - b) * (s - c));
}

float trianglePerimeter(float a, float b, float c)
{
    return a + b + c;
}

int main()
{
    float l, b;
    float a, c, d;

    float (*area)(float, float);
    float (*perimeter)(float, float);
```

```
    printf("Enter length and breadth of rectangle: ");
    scanf("%f %f", &l, &b);

    area = rectangleArea;
    perimeter = rectanglePerimeter;

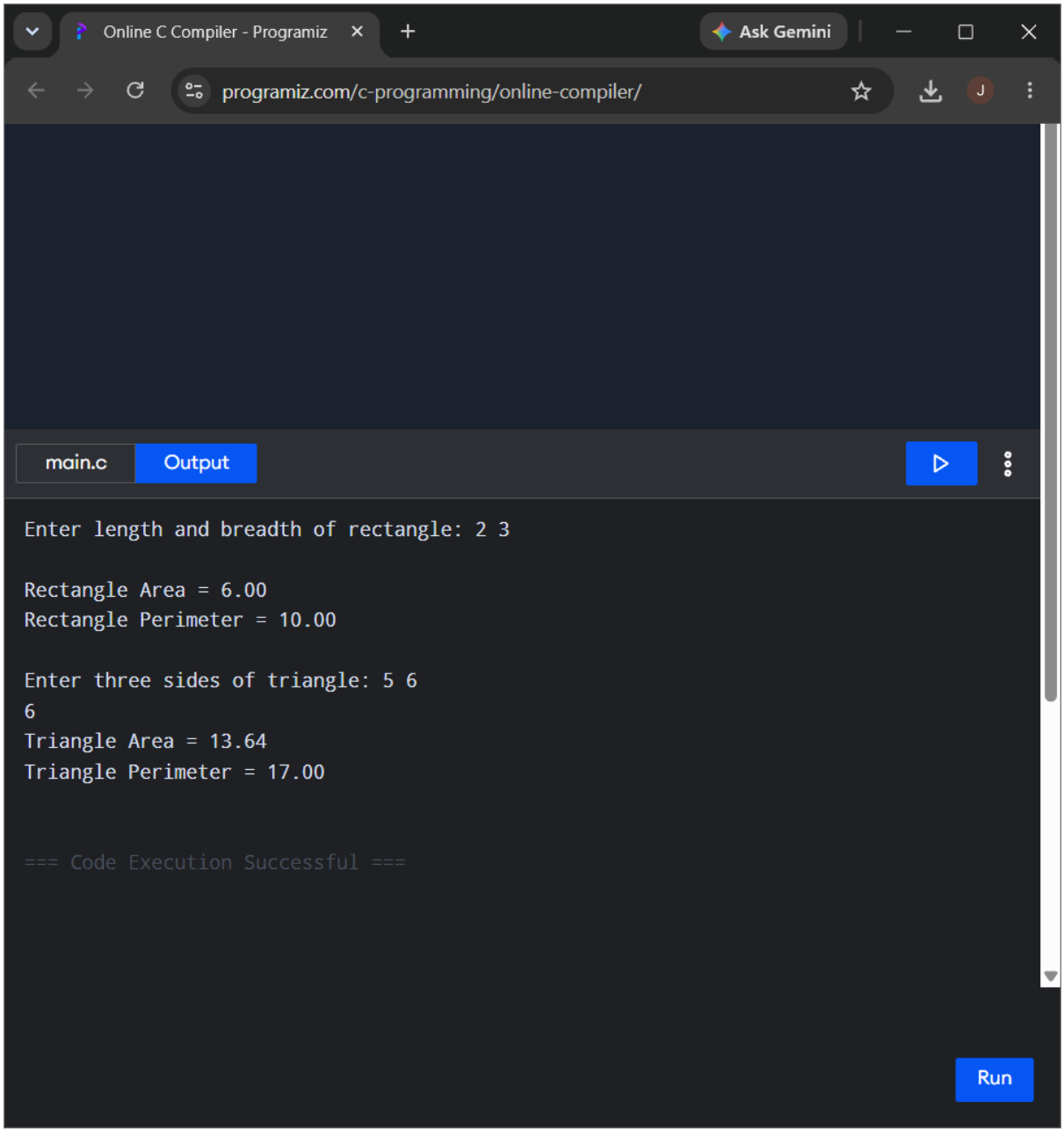
    printf("\nRectangle Area = %.2f", area(l, b));
    printf("\nRectangle Perimeter = %.2f\n", perimeter(l, b));

    printf("\nEnter three sides of triangle: ");
    scanf("%f %f %f", &a, &b, &c);

    printf("Triangle Area = %.2f",
        triangleArea(a, b, c));

    printf("\nTriangle Perimeter = %.2f\n",
        trianglePerimeter(a, b, c));

    return 0;
}
```


Online C Compiler - Programiz
Ask Gemini
programiz.com/c-programming/online-compiler/
main.c
Output
Enter length and breadth of rectangle: 2 3
Rectangle Area = 6.00
Rectangle Perimeter = 10.00
Enter three sides of triangle: 5 6
6
Triangle Area = 13.64
Triangle Perimeter = 17.00
=== Code Execution Successful ===
Run